Report

Albina Klepach

Last updated: August 15, 2024

Highlights:

- the code for AD was prepared and showed loss decay and accuracy > 95%
- the reward predictor was added to AD and tested (however haven't showed significant improvements)
- evaluation showed the stable ability to in-context learn on goals from training dataset, however, no stable ability to in-context learning on test data (probably due to overfitting to training data)
  - 40 goals * 300 traj * 2000 steps / (128 batch_size * 60 seq_len) ≈ 3000 updates ← too less for the selected model! and not enough for smaller model, so, perhaps, more data is needed for training.
- thanks to colab for Tesla T4 ^-^

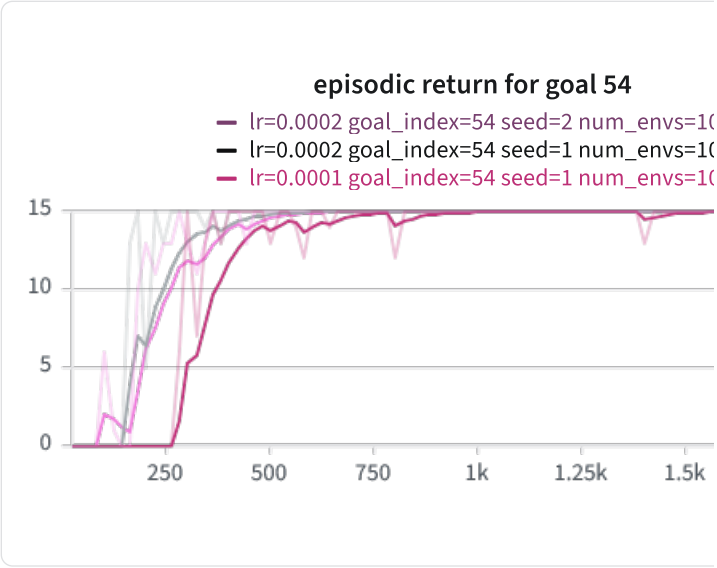
Stage 1. Data collection.

For the environment for all experiments was chosen DarkRoom with the following params:

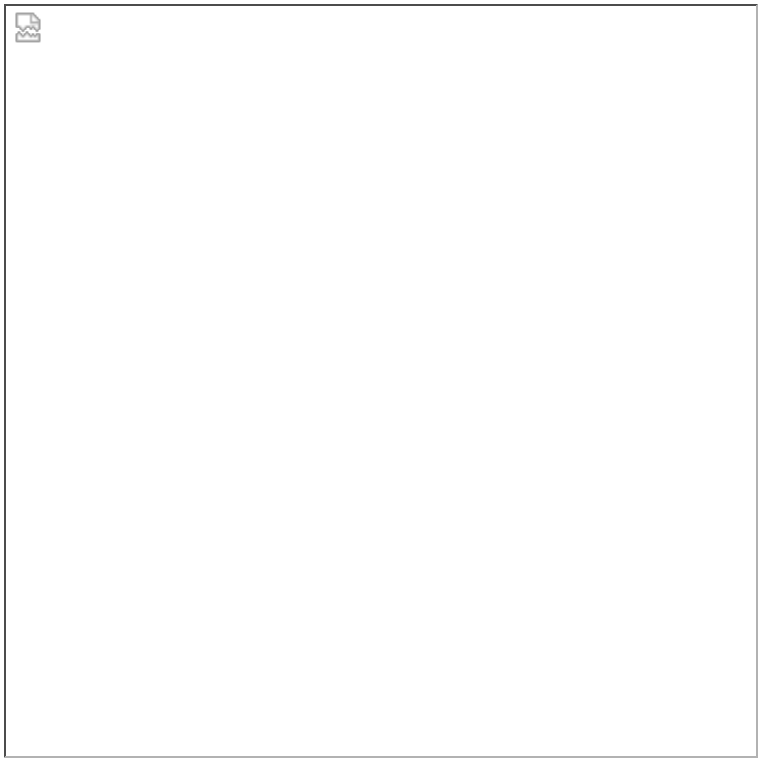
```
DarkRoom params:
  size: 9
  max_episode_lenght: 20
  random_start: false
  terminate_on_goal: false
```

For data collection I decided to choose PPO (clean rl implementation).

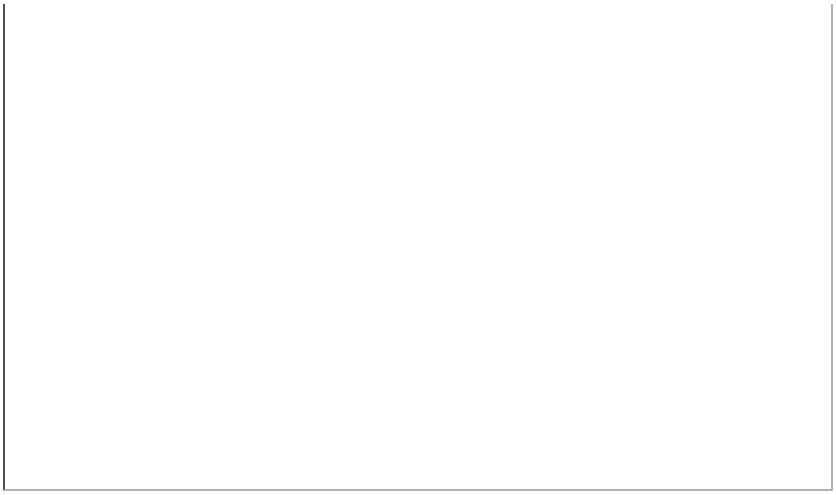
For each goal the algorithm was trained 3 times on 100 envs resulting in 300 learning trajectories of 2000 steps.

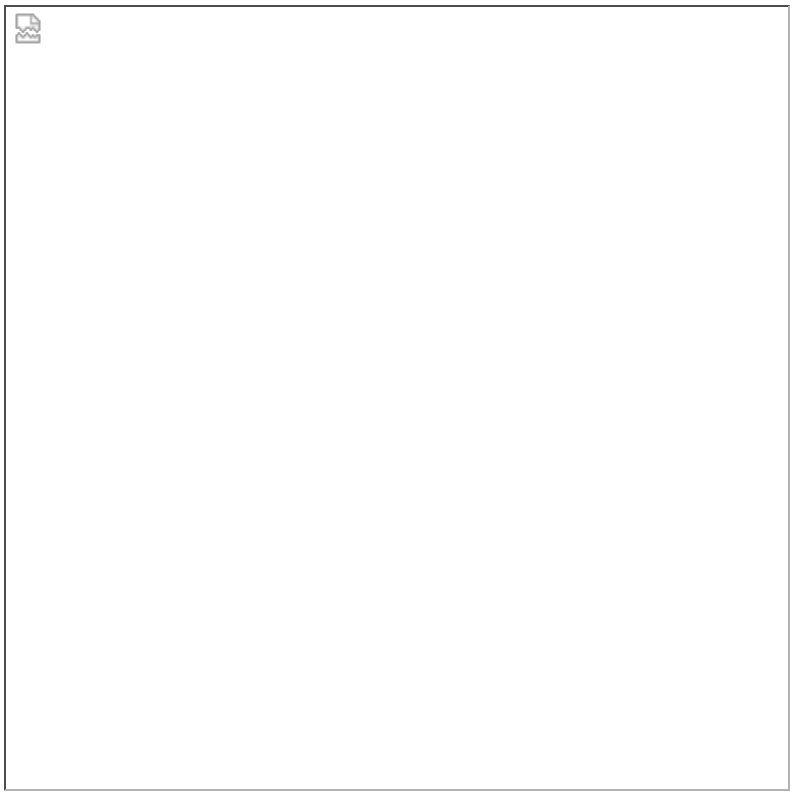


The mean curves over all goals and actors for each run:









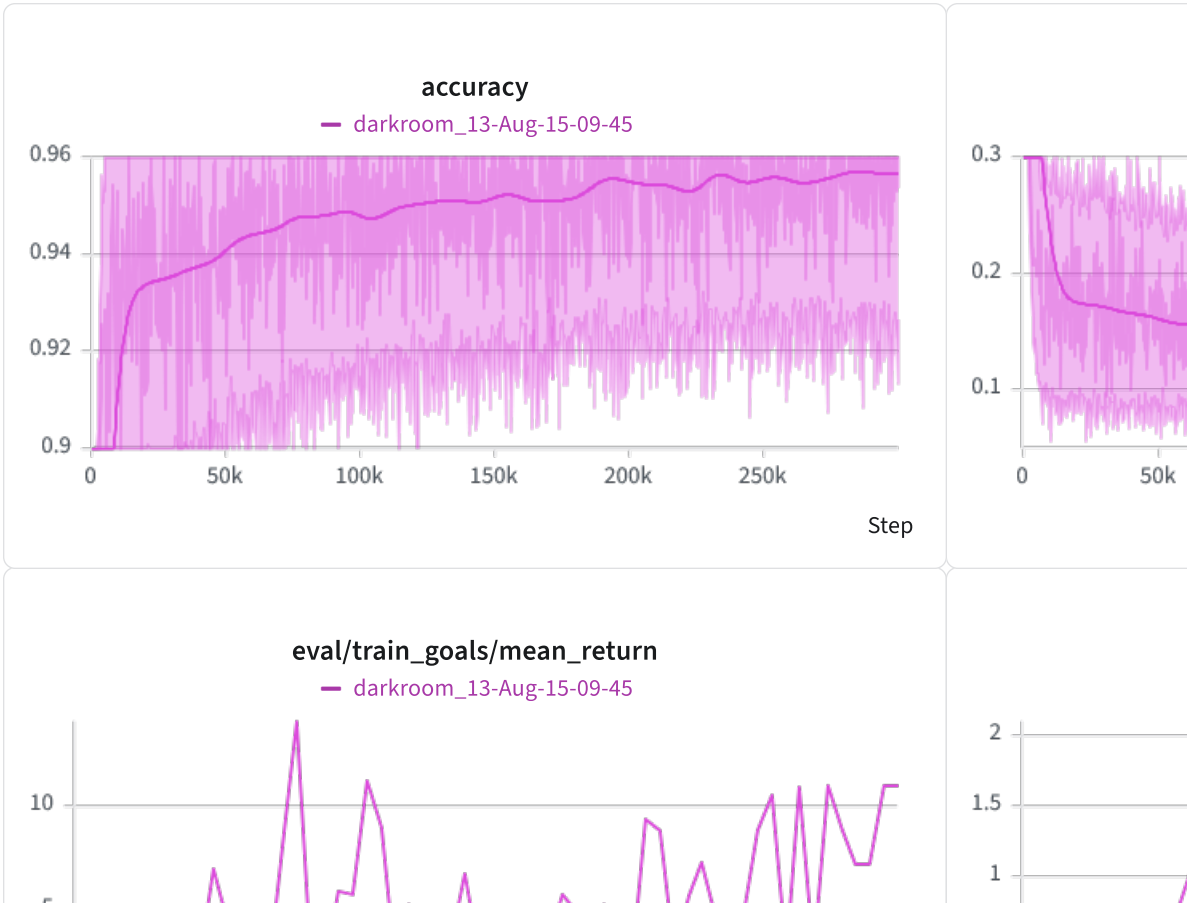
Stage 2. LLM training.

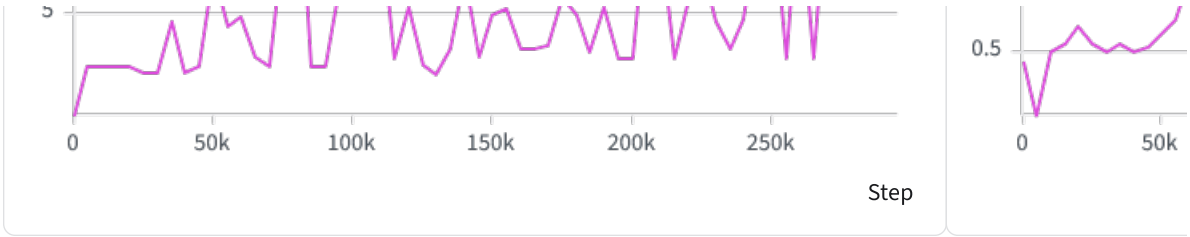
The Decision Transformer was selected as a base model for AD.

```
Params:
  # --- optimizer ---
  learning_rate: 0.0003
  batch_size: 128
  betas:
```

```
- 0.9
- 0.999
clip_grad: 1.0
warmup_steps: 10000
weight_decay: 0.001
num_updates: 300000
# --- decision transformer ---
seq_len: 60
embedding_dim: 64
hidden_dim: 256
ln_placem: postnorm
attention_dropout: 0.5
embedding_dropout: 0.1
residual_dropout: 0.1
num_heads: 4
num_layers: 4
# --- data ---
permutations_file: saved_data/permutations_9.txt
train_test_split: 0.5
```

Longest run:





Testing of hypothesis:

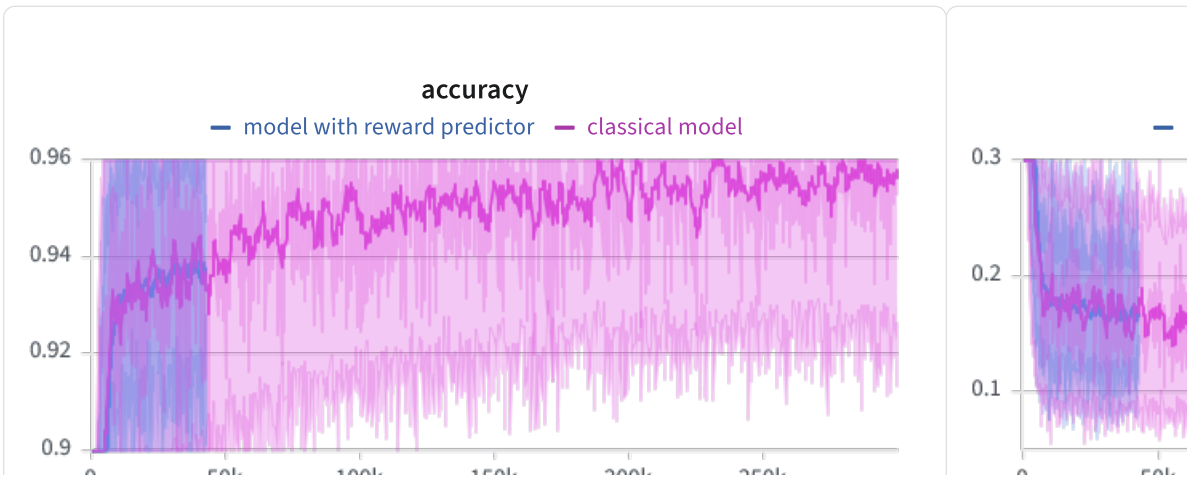
Currently during the transformer training we condition only future actions during backprop, however, it may be hard for a model to understand the dynamics of the environment, which might be quite important on tasks with complex dynamics, so we can try to use the provided dataset of trajectories to learn the state transition function and/or (easier) the reward function.

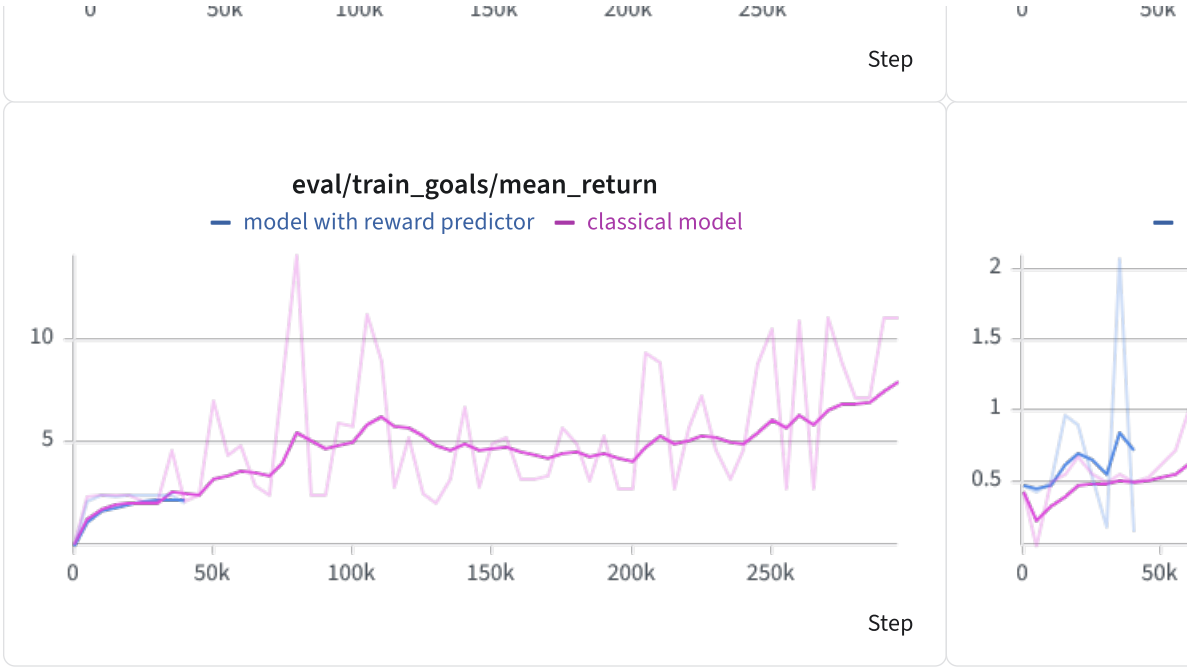
To start checking the hypotheses of the importance on the environment dynamics on the provided task I have added a reward predictor to Decision Transformer, which it trained along side with the action predictor. The evaluation stage remains unchanged in this experiment.

The idea is motivated by the success of model-based RL, where the model is learning full dynamics of the environment. Probably something similar can be applied to enhance in-context learning.

Scheme of adding the reward predictor.

Compare to the experiment with reward predictor (loss on the chart is only the loss from actor's head):





Stage 3. Evaluation.

Evaluation of the classical model:

Evaluation (mode of outputs distibution) of a classical AD model.

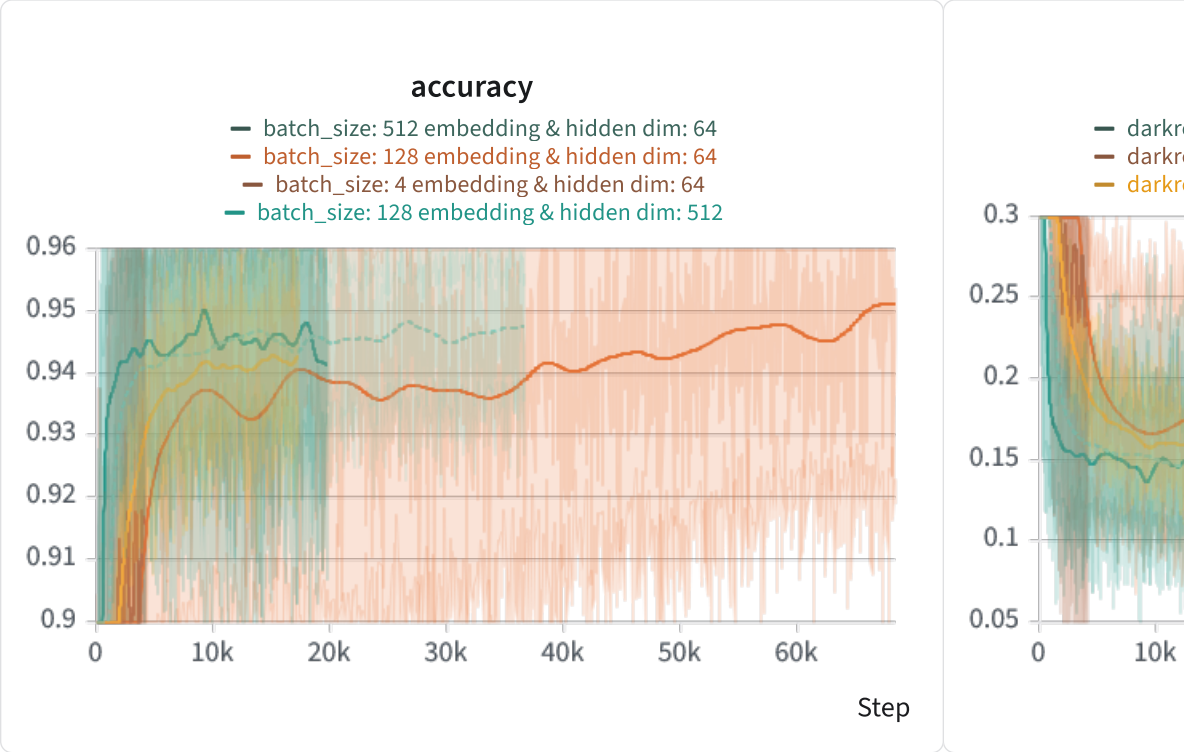
Evaluation (sampling of outputs distribution) of a classical AD model.

In both settings of evaluation we can see that training goals can be achieved for most of the goals, however, test goals can not be stably achieved. They are achieved sometimes but then can be suddenly forgotten.

Evaluation of a model with reward predictor:

Evaluation (sampling of outputs distibution) of a model with reward predictor after 40k steps.

Some other not finished experiments (these experiments had a bug in evaluation so I didn't put plots of evaluation here):



https://forge.coreweave.com/wandb/albinakl/AD/reports/Report--Vmlldzo5MDM3NDM2?accessToken=slmwe36aqxke4n76bch02mtgwnejeexvhtuc6zw4zpt3qurea3emzdwhqby20snm

Made with Weights & Biases. **Sign up** or **log in** to create reports like this one.